

1. (b) Differentiate between microcontroller and SoC.

Feature	Microcontroller Unit (MCU)	System on Chip (SoC)
Architectural Complexity	Simpler architecture, built around a microcontroller core (e.g., ARM Cortex-M). Typically lacks a Memory Management Unit (MMU).	Complex architecture, built around a high-performance multi-core processor (e.g., ARM Cortex-A) and includes an MMU.
Component Integration	Integrates CPU, Flash memory, SRAM, and basic peripherals on a single chip.	Integrates CPU, hardware accelerators (GPU, NPU, DSP), Memory Controller, and advanced peripherals on a single chip. Memory is often external.
Performance & Power	Operates in the milliwatt range. Prioritizes low-latency, deterministic, and ultra-low-power operations.	Operates in the watt range, requiring thermal management. Prioritizes high computational throughput and multimedia processing.
Operating System Support	Runs bare-metal applications or Real-Time Operating Systems (RTOS) like FreeRTOS.	Hosts full-fledged Embedded Operating Systems like Embedded Linux or Android Automotive OS.
Application Domain	Engine Control Units (ECUs), smart sensors, simple home appliances.	Advanced Driver Assistance Systems (ADAS), infotainment systems, complex IoT gateways.

3. (b) Differentiate between general-purpose computing systems and embedded systems.

Feature	General-Purpose Computing System	Embedded System
System Purpose	Versatile and flexible; designed to handle an unknown variety of future workloads defined by the user.	Application-specific and fixed-function; designed to execute a single or tightly coupled set of predefined tasks.
Operating System	General-Purpose Operating System (GPOS) like Windows or standard Linux, optimized for fairness and average throughput.	Real-Time Operating System (RTOS) or bare-metal loops, optimized for predictability and hard real-time deadlines.
Hardware Architecture	Utilizes high-performance microprocessors with massive external RAM, secondary storage, and standard interfaces.	Built around a microcontroller or an application-specific SoC with integrated memory and custom peripherals to minimize Size, Weight, Power, and Cost (SWaP-C).
Software Flexibility	Software is easily loaded, modified, or updated by the end-user.	Software (firmware) is often burned into non-volatile memory during manufacturing and is rarely altered by the end-user.
Environmental Operation	Typically operates in controlled, benign environments (offices, server rooms).	Often operates in harsh physical environments subject to extreme temperatures, vibration, and electromagnetic interference (EMI).

4. (a) Interpret Emulation and explain how it differs from pure software simulation.

Feature	Pure Software Simulation	Emulation
Execution Medium	Software executing on a general-purpose host PC.	Hardware executing on specialized substitute hardware, typically Field Programmable Gate Arrays (FPGAs).
Accuracy & Fidelity	Functionally accurate (e.g., Instruction Set Simulators) but often abstracts away complex bus behaviors and precise memory latencies.	Cycle-exact accuracy; accurately replicates bus cycles, race conditions, memory access times, and precise interrupt latencies.
Execution Speed	Can be slower than real-time due to the overhead of simulating hardware behavior in software.	Executes significantly faster than software simulation, often approaching real-time speeds due to hardware execution.
External Interfacing	Difficult to interface directly with physical external signals or hardware in real-time.	Can interface directly with physical external signals, making it suitable for low-level driver development and hardware-software integration.
Primary Use Case	Early-stage algorithm verification, logic testing, and high-level software development before hardware is available.	Deep, cycle-accurate visibility into hardware-software interaction at the chip level; low-level debugging and validation.